



FloodLens — Ground-Up Execution Plan

Multimodal AI Hackathon 2026 · Track B

Build the core first. Polish later. Win.

⚠ THE GOLDEN RULE: Do NOT start Streamlit, the report, Nepali translation, or any dashboard work until the classical flood pipeline produces a working **flood_mask.tif** over the Trishuli valley. The three core technical criteria together account for 75% of the score, so build the core before polishing the UI.

1. Build Priority — The Exact Order to Follow

The evaluation weights tell you exactly what to build first. Never invert this order.

#	What to Build	Priority	Score
1	SAR change detection → flood_mask.tif <i>Preprocess Sentinel-1, create a SAR change candidate, then fuse optical evidence. Core of mapping.</i>	CRITICAL	Shared 30%
2	Infrastructure damage overlay <i>Intersect flood mask with OSM buildings, roads, bridges.</i>	HIGH	Shared 30%
3	Cut-off settlement analysis <i>OSMnx graph → remove flooded edges → Dijkstra → no path = cut off.</i>	HIGH	20 %
4	U-Net flood segmentation + MC Dropout uncertainty <i>Train on Kuro Siwo, EfficientNet-B0 encoder, uncertainty heatmap.</i>	HIGH	25 %
5	Dashboard + auto situation report (English + Nepali) <i>Streamlit + Folium. ALL numbers from pipeline — none invented.</i>	NORMAL	10 %
6	Report + Q&A + limitations; DEM tracing is optional bonus <i>6-page report, EMSR927 checking, 3-min video; optional pysheds D8 demo.</i>	HIGH	15% (+ bonus)

2. What to Do Right Now — Your First 14 Steps

Follow these in exact order. Do not skip ahead. Step 14 is your first technical checkpoint; final M1 comes after Sentinel-2 fusion.

Step	Action	Priority
1	Create the GitHub repository + folder structure <i>FloodLens/ with src/, data/raw/, app/, notebooks/, outputs/ sub-folders. Push an empty README.</i>	CRITICAL
2	Create Python virtual environment <i>python -m venv floodlens-env and activate it. Keep it clean.</i>	CRITICAL
3	Install the core geospatial stack (first batch only) <i>pip install numpy pandas matplotlib rasterio geopandas shapely scipy scikit-image folium streamlit osmnx networkx</i>	CRITICAL
4	Register on Copernicus Dataspace <i>dataspace.copernicus.eu — approval can take 24 h. Do this TODAY.</i>	CRITICAL
5	Define the Trishuli AOI bounding box <i>Hard-code EVENT_DATE='2026-08-26' and a lat/lon bbox around the Bhote Koshi–Trishuli corridor. ~85.1E–85.8E, 27.8N–28.3N.</i>	CRITICAL
6	Search for Sentinel-1 BEFORE image <i>Use sentinelSAT or the Copernicus Browser UI. IW mode, VV+VH polarisation. Date: ~5–15 days before Aug 26.</i>	CRITICAL
7	Search for Sentinel-1 AFTER image <i>Same orbit track (same relative orbit number) as the BEFORE image. Date: ~5–15 days after Aug 26. 12-day repeat cycle.</i>	CRITICAL
8	VERIFY same orbit track <i>Check relativeOrbitNumber in both image metadata. If they differ, do NOT compare them — find a pair that match.</i>	CRITICAL
9	Download both Sentinel-1 GRD products <i>Store the raw BEFORE and AFTER products under data/raw/sentinel1/before/ and /after/. Do not compare them until both are preprocessed identically.</i>	CRITICAL
10	Pull pre-event OSM snapshot <i>ohsome API: snapshot date 2026-07-27 (latest pre-event). Query buildings, roads (highway=*), bridges. Save as GeoJSON.</i>	HIGH
11	Preprocess both Sentinel-1 images consistently <i>In SNAP: Apply Orbit File → radiometric calibration → optional speckle filtering → terrain correction. Export comparable linear-backscatter or dB GeoTIFFs.</i>	CRITICAL
12	Align and verify the processed rasters <i>Reproject the AFTER raster to the BEFORE raster's CRS and pixel grid if needed. Load with rasterio; verify CRS, transform, shape, and display side-by-side.</i>	CRITICAL
13	Compute the SAR change image <i>If inputs are linear intensity: change = log(after / before). If inputs are already in dB: change = after_dB - before_dB. Never take a logarithm of dB values.</i>	CRITICAL
14	Apply Otsu → save SAR flood/change candidate <i>Threshold the SAR change metric and save flood_mask_sar.tif. Treat it as a candidate mask, not final flood truth. ← TECHNICAL CHECKPOINT</i>	CRITICAL

✓ **TECHNICAL CHECKPOINT** reached when flood_mask_sar.tif shows a plausible change corridor along the Trishuli valley. Next complete Sentinel-2 + mask fusion in Module 1 to create the final flood_mask.tif before downstream modules.

3. Module-by-Module Implementation Guide

Module 1 SAR Change Detection + NDWI Merge [Mapping criterion — shared 30%]

1A — SAR Preprocessing + Change Detection

- Input: raw Sentinel-1 GRD BEFORE and AFTER products from the same orbit track; use the same polarisation(s) and processing settings.
- Preprocess both consistently: Apply Orbit File → radiometric calibration → optional speckle filtering → terrain correction. Export either linear backscatter or dB, but use the same scale for both.
- Align to one pixel grid. For linear values use $\log(\text{after} / \text{before})$; for dB values use $\text{after_dB} - \text{before_dB}$. Otsu can then threshold the change image.
- Output: flood_mask_sar.tif — a SAR-derived flood/change candidate mask, not yet the final flood mask.

1B — Sentinel-2 Water + Debris Supplement

- Download pre-event and post-event Sentinel-2 L2A when cloud cover is acceptable. Band 3 = Green, Band 8 = NIR.
- Compute $\text{NDWI} = (\text{Green} - \text{NIR}) / (\text{Green} + \text{NIR})$ and compare before vs after. Use ~0.3 only as a starting threshold, then tune from the imagery. Add spectral-change evidence for exposed mud/debris because NDWI mainly captures open water.
- Output: flood_mask_optical.tif for water plus an optical debris/change layer when reliable.
- If cloud-covered: skip unreliable optical pixels or the optical branch entirely. Do not force a bad Sentinel-2 image.

1C — Fuse Evidence into the Final Flood Mask

- Fuse the SAR candidate with reliable optical water and debris/change evidence. Apply basic morphological cleanup and, where useful, DEM/slope context to suppress obvious terrain-shadow or layover false positives.
- Do not interpret every SAR change pixel as flood/debris: SAR change may also come from shadow, layover, vegetation change, or noise.
- Output: flood_mask.tif ← final fused mask that feeds every downstream module

⚠ **DISQUALIFICATION RISK:** Never use EMSR927, UNOSAT, or any published damage map as input. These are for checking your results only. OSM edits made after Aug 26 are also forbidden as inputs.

Module 2 Infrastructure Damage Detection [Mapping criterion — shared 30%]

- Load pre-event OSM layers (ohsome snapshot 27 Jul 2026): buildings, roads (highway=*), bridges.
- Convert flood_mask.tif to vector polygons using rasterio.features.shapes → flood_polygons.geojson
- Spatial join with geopandas: building/road intersects flood polygon → 'potentially_damaged = True'
- Output: damaged_buildings.geojson, damaged_roads.geojson
- Compute summary statistics: count of damaged buildings, km of road disrupted, bridges hit.

- These numbers feed directly into the situation report — do not invent them.

Note: 'potentially damaged' is an honest label — your system estimates overlap from satellite data, not ground truth. Use this exact language in the report. It satisfies the honesty criterion.

Module 3 Cut-off Settlement Analysis [20% of Total Score]

- Build the road network graph with OSMnx from the pre-event OSM snapshot (same 27 Jul 2026 date).
- Identify all road edges that intersect damaged_roads.geojson.
- Remove those edges from the NetworkX graph: `G.remove_edges_from(disrupted_edges)`
- Identify settlement nodes (OSM place=village/hamlet/town) and hospital/town nodes.
- For each settlement: `nx.shortest_path(G, settlement, nearest_hospital)`
- If `NetworkXNoPath` is raised → settlement is CUT OFF. Append to `cutoff_settlements.geojson`
- Output: `cutoff_settlements.geojson` with settlement name, coordinates, and reason (which road segment is broken)

This is the highest-impact feature in your demo. When you show a judge that Village X can no longer reach the district hospital because Bridge Y was swept away, that is the moment the system feels real. Build it carefully.

Module 4 AI Flood Segmentation + Uncertainty [25% of Total Score]

4A — Dataset Setup

- Download Kuro Siwo dataset (MIT License — cite Bountos et al., 2024).
- Split by flood event, not random tiles: for example 8 events for training, 2 for validation, and 2 strictly held-out labeled test events. Keep the Trishuli case study fully unseen until final evaluation.
- Input channels: VV and VH polarisation (2-channel SAR input tensor).
- Kuro Siwo provides binary flood labels for model development. Do not use EMSR927 as a training input, tuning label, or checkpoint-selection signal.

4B — Model Architecture

- U-Net with EfficientNet-B0 encoder (pretrained on ImageNet).
- Use the segmentation-models-pytorch library: `import segmentation_models_pytorch as smp`
- `model = smp.Unet(encoder_name='efficientnet-b0', in_channels=2, classes=1, activation='sigmoid')`
- Loss: $0.5 * BCELoss + 0.5 * DiceLoss$ — handles class imbalance between flooded and dry pixels.

4C — Uncertainty with Monte Carlo Dropout

- Add Dropout layers ($p=0.1$) in the decoder.
- At inference, keep `model.eval()` so BatchNorm remains fixed, then selectively switch only Dropout modules to training mode.
- Run 20 stochastic forward passes through the same trained model on the same input tile.
- Mean across 20 passes → predicted flood probability map.
- Standard deviation across 20 passes → pixel uncertainty map (save as `uncertainty_map.tif`).

- High uncertainty pixels = 'requires ground verification'. Show this on the dashboard as a yellow/red heatmap overlay.

4D — Metrics to Report

- Compute IoU, F1-score, precision, and recall on held-out labeled Kuro Siwo test events.
- For Himalayan generalization, run the frozen model on the Trishuli case study first; only afterward compare its output with EMSR927 as a checking reference.
- Report Trishuli overlap metrics separately, plus Expected Calibration Error (ECE) or another uncertainty-calibration diagnostic.
- Never train, tune thresholds, select checkpoints, or otherwise optimize the model using EMSR927. It is validation/checking data only.

Module 5 Dashboard + Situation Report [10% — Build Last]

- Framework: Streamlit. One script: app/app.py
- Sidebar inputs: bounding box (lat/lon), flood date, Run Pipeline button.
- Map panel: Folium map with toggleable layers:
 - Flood mask (blue fill)
 - Uncertainty heatmap (yellow → red gradient, opacity slider)
 - Damaged buildings (orange polygons)
 - Damaged roads (red lines)
 - Cut-off settlements (red markers with popup showing settlement name + why it is cut off)
- Situation report: pull all numbers from pipeline output files. Template fills them in. Zero LLM hallucination.
- Nepali translation: Helsinki-NLP MarianMT (opus-mt-en-ne). Runs offline. No API key.
- 'System Limitations' panel: always visible in the UI sidebar — Sentinel revisit gap, no real-time warning, OSM incompleteness.
- PDF export: WeasyPrint or ReportLab. One button.

Module 6 Bonus: DEM Flood Path Tracing [Optional Bonus — separate from the 15% criterion]

- Library: pysheds
- Input: Copernicus WorldDEM-30 GeoTIFF covering the corridor.
- Process: fill sinks → compute D8 flow direction → trace flow accumulation downstream from any upstream point.
- Intersect the traced path with OSM settlement polygons → list settlements in the flood path.
- Output: flood_path.geojson, list of at-risk settlements with distance from source.
- Keep this optional until the scored core is complete. The D8 path trace is visually strong in a live demo, but it does not replace the 15% report/Q&A/limitations criterion.

4. Rules You Cannot Break — Disqualification Risks

Rule	Why it matters
Never use EMSR927, UNOSAT, or any published damage map as INPUT	Using these as inputs → immediate disqualification. They may only be used to validate your results.
Never use post-event OSM edits as input	OSM was updated after the flood with damage data. Use only the 27 Jul 2026 ohsome snapshot.
Same orbit track for Sentinel-1 pairs	Different orbit tracks view terrain from different angles in mountains. Pixel-by-pixel comparison is invalid.
Every number in the situation report must come from your pipeline	No LLM can invent statistics. If your pipeline says 47 buildings were hit, the report says 47. Not 'dozens'.
No images of victims in demo or report	The problem statement explicitly requires respectful treatment of a recent disaster. This is an ethical requirement.
Include all required data attributions in every submitted document	Missing attribution = lost credibility with judges. Attribution text is in the problem statement verbatim.

5. GitHub Repository Structure

Create this structure on Day 1. Every module has a home before you write a line of code.

```
FloodLens/
├── data/
│   ├── raw/sentinel1/before/
│   ├── raw/sentinel1/after/
│   ├── raw/sentinel2/
│   ├── raw/dem/
│   ├── raw/osm/
│   ├── processed/
│   └── training/kuro_siwo/
├── src/
│   ├── data_ingestion/
│   ├── flood_mapping/
│   ├── ai_model/
│   ├── infrastructure/
│   ├── network_analysis/
│   └── reporting/
├── app/app.py
├── notebooks/
├── outputs/maps/
├── outputs/geojson/
├── outputs/reports/
├── requirements.txt
├── .gitignore ← add data/ to avoid committing GBs of satellite imagery
└── README.md
```

6. Python Environment — Three-Stage Install

Do NOT install everything at once. Install each batch only when you reach that module.

Stage 1 — Core Geospatial (Install on Day 1)

```
pip install numpy pandas matplotlib rasterio geopandas shapely scipy scikit-image folium  
streamlit osmnx networkx sentinelsat
```

Stage 2 — AI / Deep Learning (Install when starting Module 4)

```
pip install torch torchvision segmentation-models-pytorch albumentations
```

Stage 3 — Reports and Translation (Install when starting Module 5)

```
pip install pysheds transformers sentencepiece reportlab
```

7. The Five Milestones That Define Readiness

M1	flood_mask.tif	Final flood_mask.tif combines the SAR candidate with reliable optical evidence where available and shows a plausible flood/change corridor over the Trishuli valley.	CRITICAL
M2	damaged_*.geojson	Buildings and road segments intersecting the flood mask are flagged and exportable.	CRITICAL
M3	cutoff_settlements.geojson	At least one settlement is correctly identified as cut off with a named broken road segment.	CRITICAL
M4	predicted_mask.tif + uncertainty_map.tif	U-Net is evaluated on held-out labeled test events with IoU/F1 reported; the frozen model also runs on Trishuli, and uncertainty_map.tif shows meaningful variance.	HIGH
M5	Working Streamlit app	Pipeline runs end-to-end from bounding box + date input, map renders, situation report downloads in English and Nepali.	HIGH

8. Live Evaluation — Q&A You Will Definitely Face

The judges evaluate live on an area and date they choose. Prepare these answers until they are reflexive.

"What can your system NOT do?"

Sentinel revisits every 5–12 days, so we cannot warn of sudden collapses in real time. Rural OSM may omit roads or bridges, the 30 m DEM misses narrow terrain, and global-event training may transfer imperfectly to Himalayan debris floods. This is an educational prototype, not an operational tool.

"How did you validate your flood mask?"

We report IoU, F1, precision, and recall on held-out labeled Kuro Siwo events. Then we freeze the model, run Trishuli, and only afterward compare against EMSR927. EMSR927 is never used for training or tuning.

"Why MC Dropout instead of an ensemble?"

MC Dropout gives pixel-level uncertainty from 20 stochastic passes through one trained model. We keep BatchNorm in evaluation mode and activate only Dropout. It is cheaper than training several separate models; ensemble comparison is future work.

"Why should rescuers trust your cut-off analysis?"

We do not claim certainty. Every cut-off settlement is linked to a specific removed road edge that intersects the flood mask, and we can show that evidence together with the flood-mask uncertainty.

"Your flood mask doesn't match EMSR927 here — why?"

Likely a revisit-timing mismatch: Sentinel-1 may observe the valley after peak water has receded, while EMSR927 may use imagery from a different pass. We report such differences as an explicit limitation.